

CSCI 5540 – Fall 26

Tutorial 1

TAO Yeyao (in place of Yangtao)

Credits to CMU Machine Learning Systems Lecture 1

Outline

- Course Information
- Group Formation
- Course Workload
- ML System Overview

Basic Information

- Instructor:
 - Prof. Hong Xu
- TA:
 - Yangtao Deng (ytdeng25@cse)
 - Chaobo Jia (cbjia25@cse)
- Course Website:
 - <https://github.com/henryhxu/CSCI5540> (updated weekly)
 - Including all materials
- Questions, Discussions, Announcements:
 - <https://edstem.org/au/join/y5we2K>
- Notes:
 - All updates will be sent to your emails. Please check your university emails.
 - If you have any questions, please ask the TAs via email or during tutorial sessions.

Group Formation

- Form a group for group-based presentation & project:
 - <https://forms.gle/QfSWsvF7NHtutVC56>
 - Groups of 5 students
 - Due Date: 23:59 Sep 13
 - Declare your group's membership and paper preferences by the due date. (papers list will be sent to you soon. Stay tuned.)
 - Students without grouping will be grouped by TA

Student Lectures

- Group presentation in each lecture (2~3 papers)
 - One group: paper presentation for at most 40 minutes
 - Another group: reviewers to engage in a constructive critique and ask questions
 - Others: free to ask questions

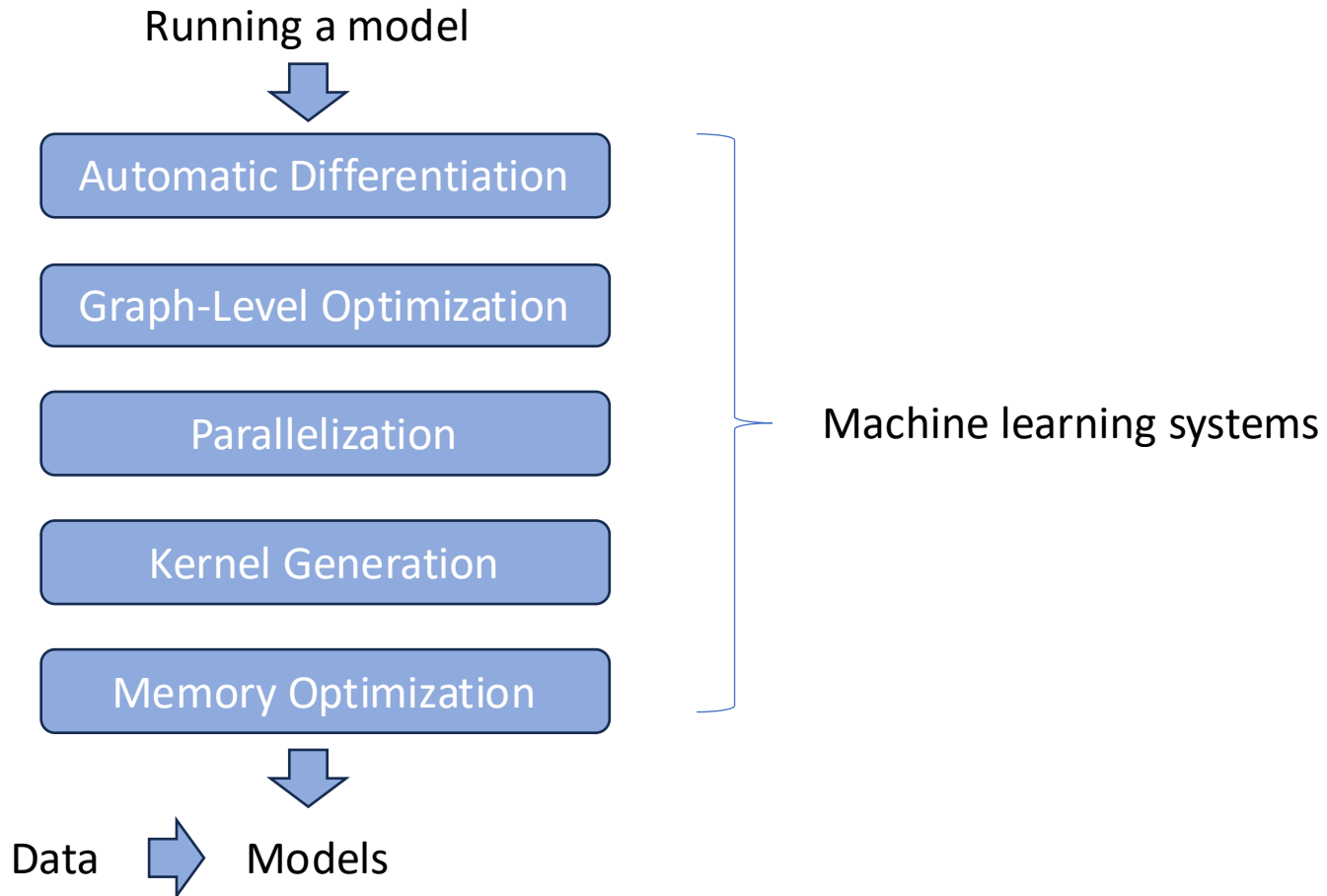
Group Project

- Email a 2-page draft proposal (including references) by September 30.
 - Remember to include the names and CUHK email addresses of the group members.
- Mid-semester progress presentation on November 2.
- An 8-page final PDF report with your source code via email
 - Due date: 1:00PM EST on December 17.
- Final project presentation.

Course Workload

• Seminar Paper Presentation	20%
• Paper Summary	10%
• Participation	10%
• Project Report	40%
• Project Presentation	20%

An Overview of Machine learning Systems



Automatic Differentiation

Running a model



Automatic Differentiation

Graph-Level Optimization

Parallelization

Kernel Generation

Memory Optimization

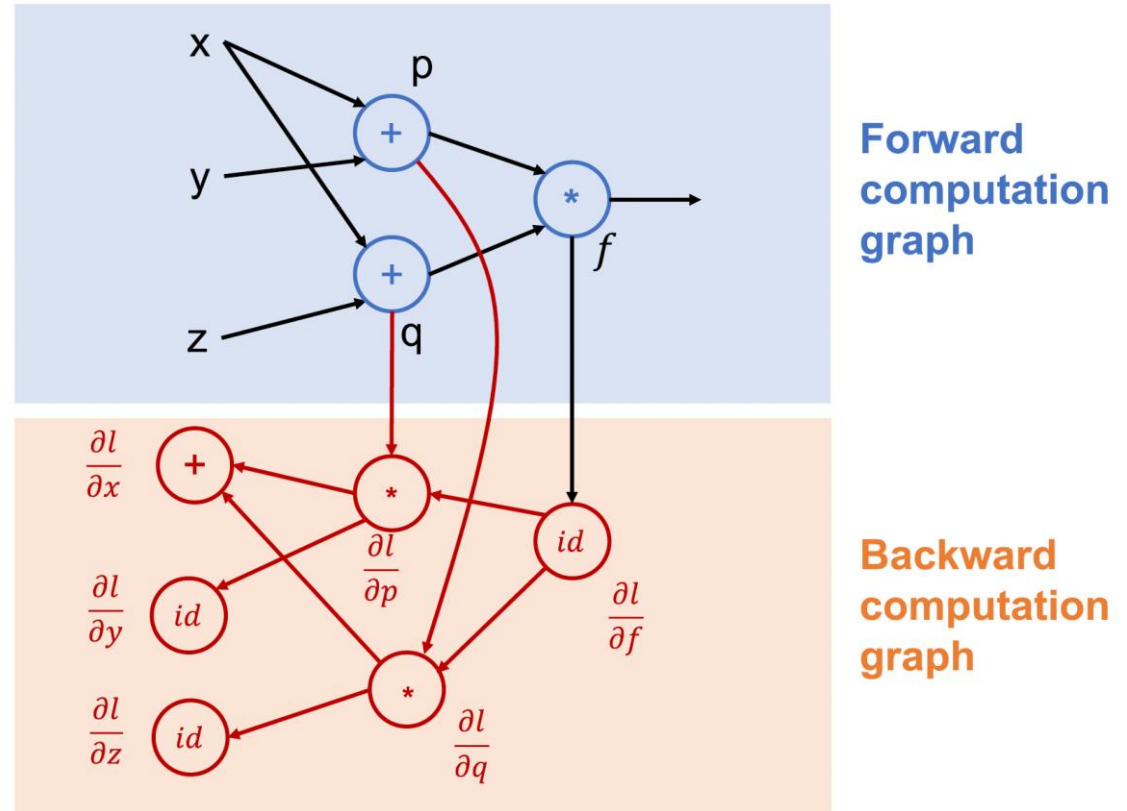


Data

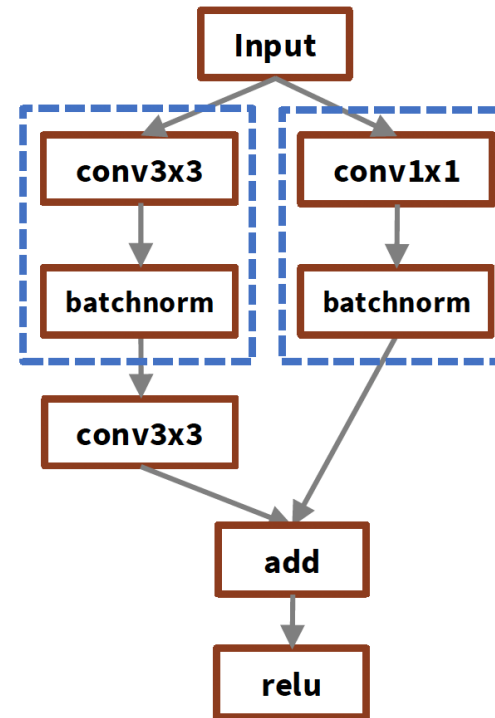
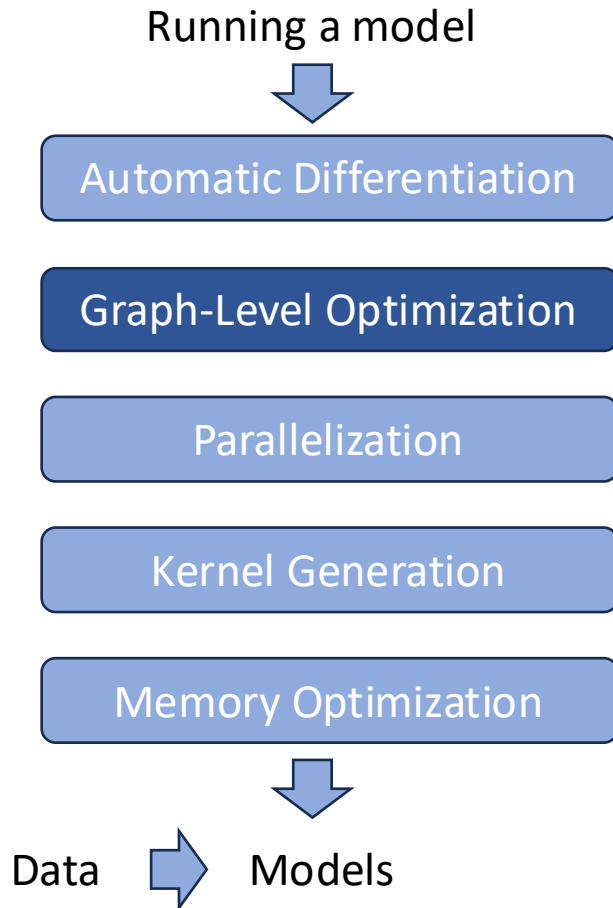


Models

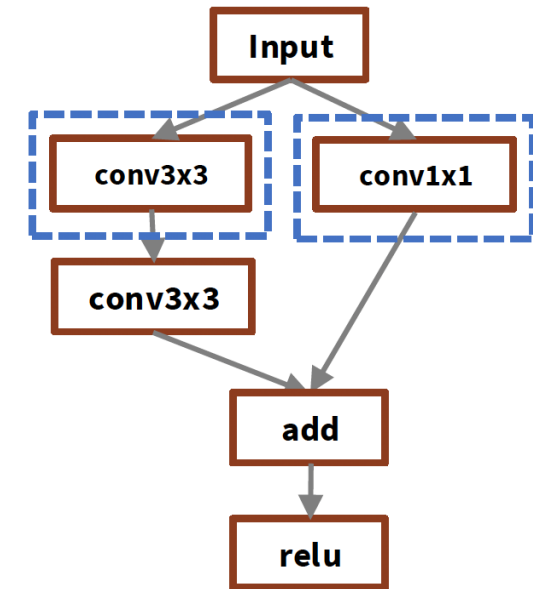
AOTAutograd: Automatically construct backward computation graph



Graph-Level Optimization

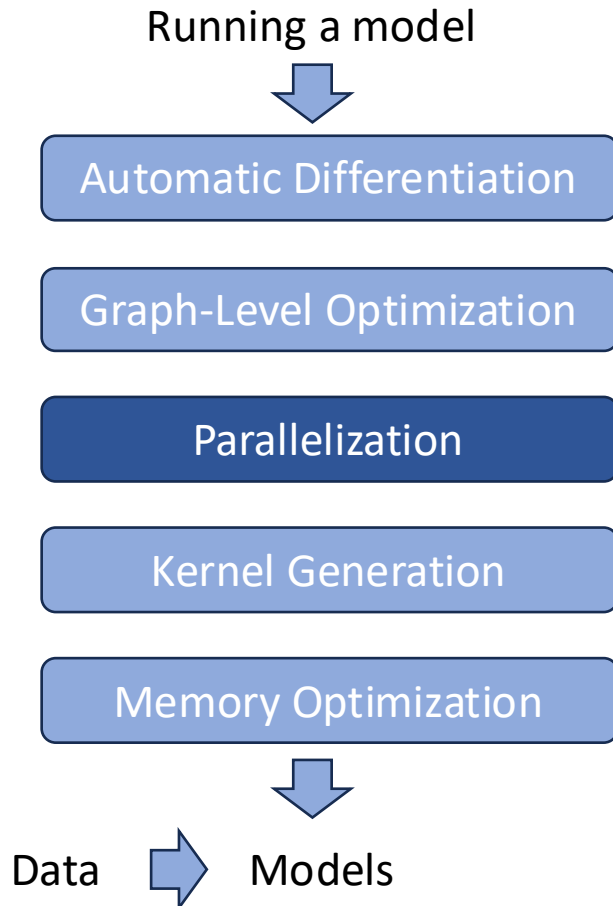


Input Computation Graph



Fused Computation Graph

Parallelization

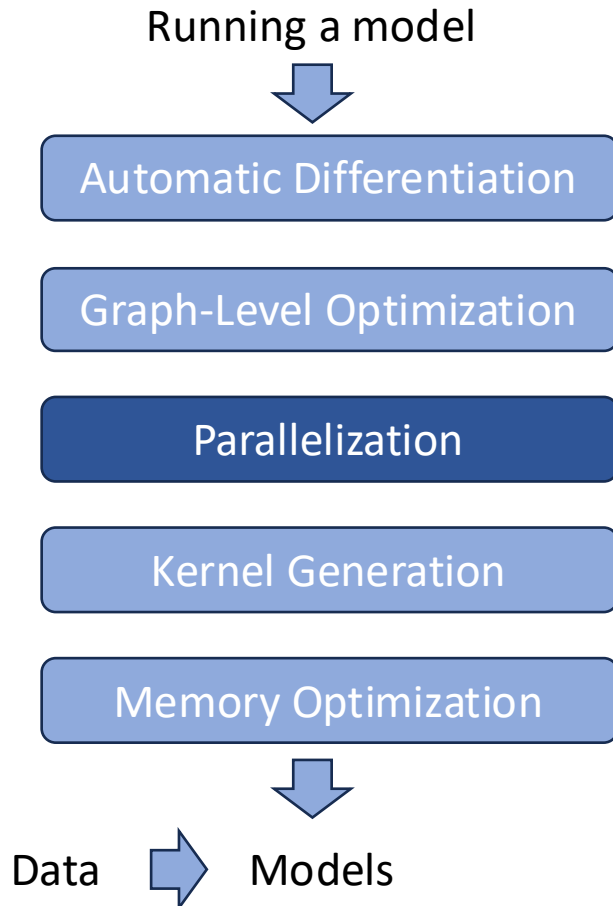


3 stages of model training

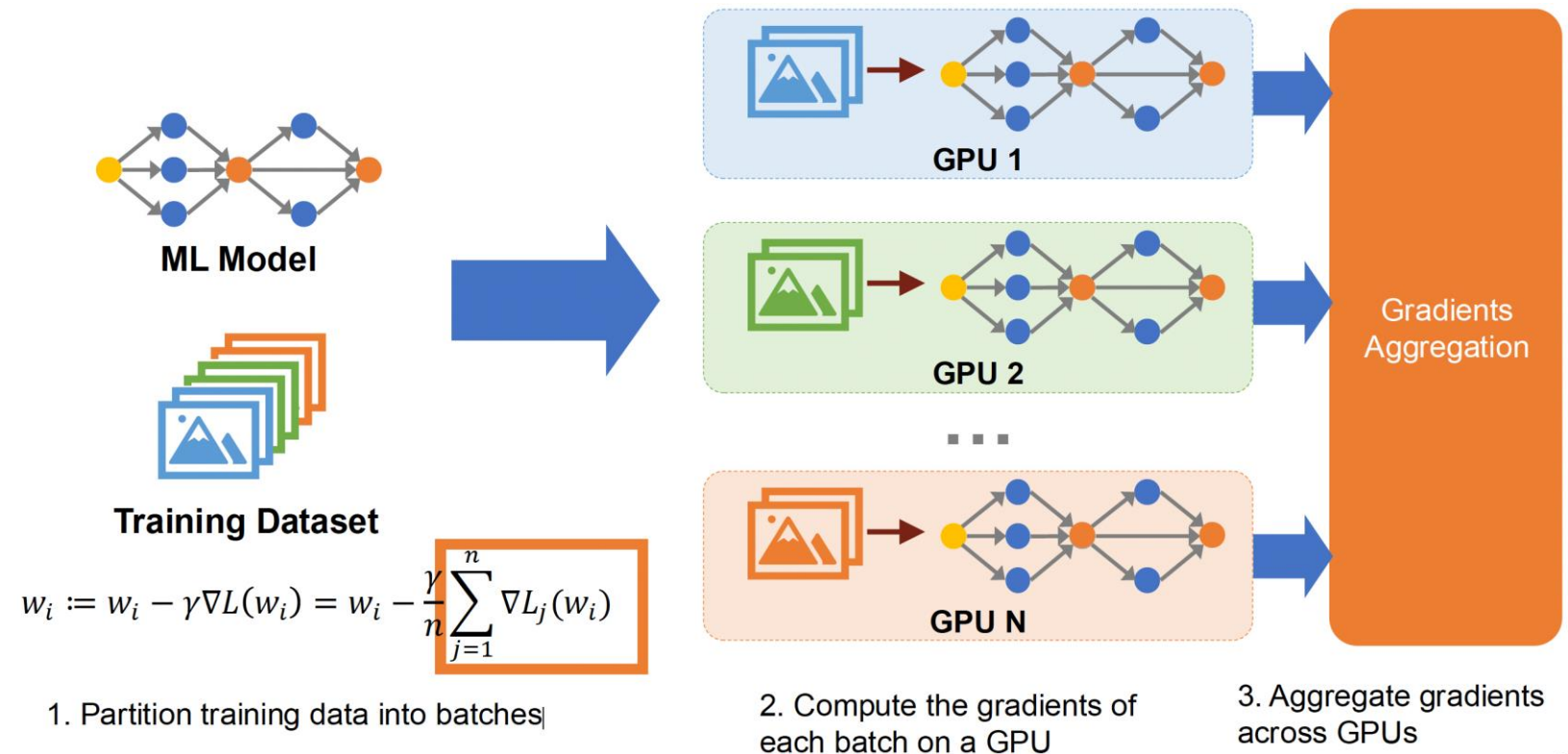
1. **Forward propagation**: apply model to a batch of input samples and run calculation through operators to produce a prediction
2. **Backward propagation**: run the model in reverse to produce error for each trainable weight
3. **Weight update**: use the loss value to update model weights

$$w_i := w_i - \gamma \nabla L(w_i) = w_i - \frac{\gamma}{n} \sum_{j=1}^n \nabla L_j(w_i)$$

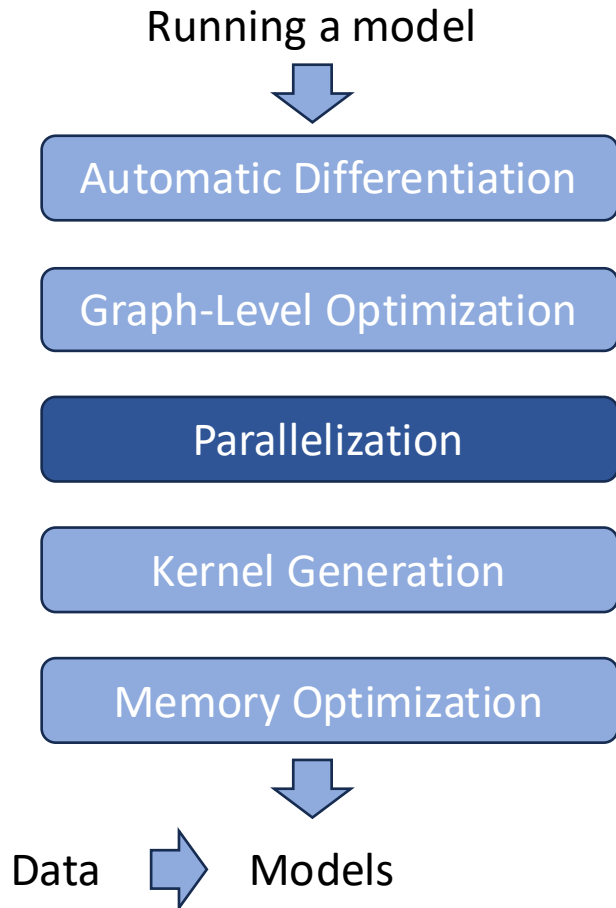
Parallelization



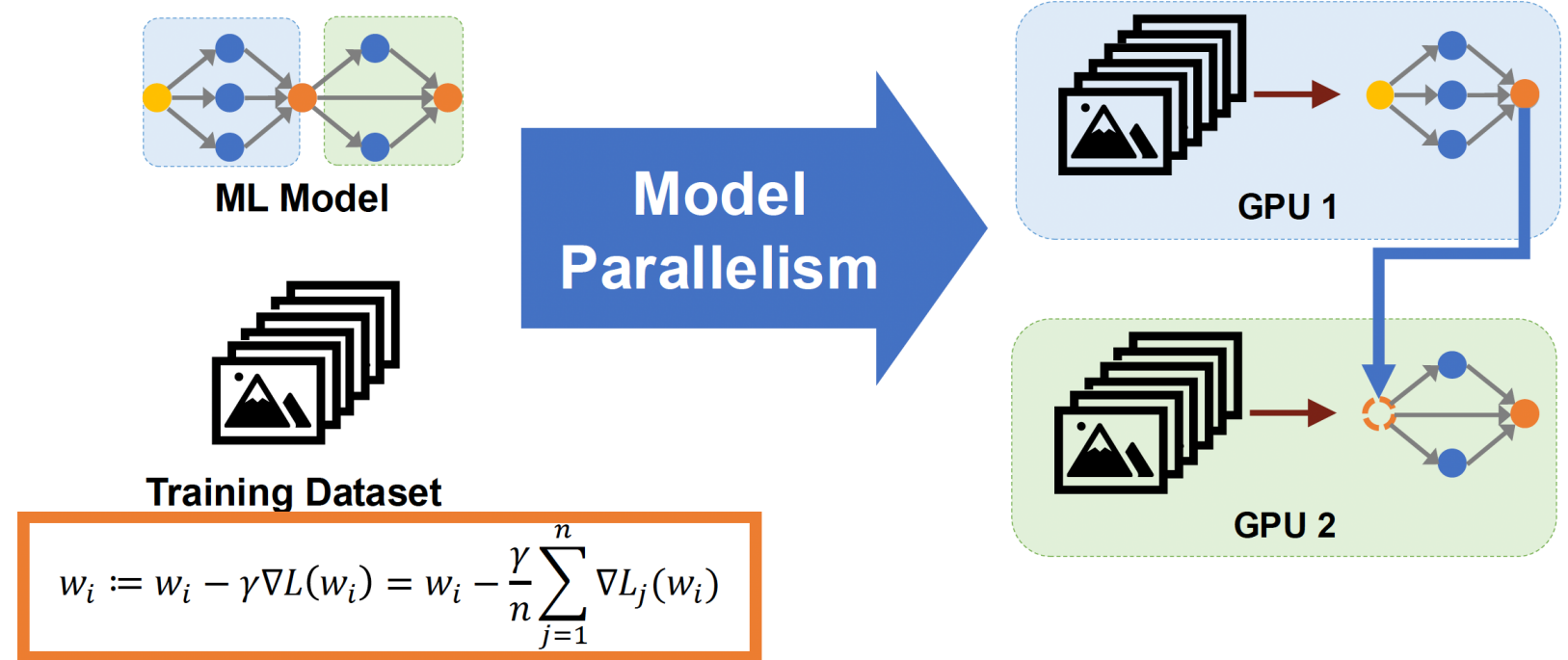
Data Parallelism: Partition training data into batches



Parallelization



Model Parallelism: Split a model into multiple chunks



Kernel Generation

Running a model



Automatic Differentiation

Graph-Level Optimization

Parallelization

Kernel Generation

Memory Optimization

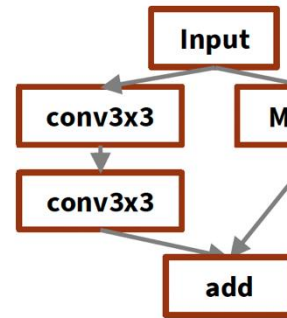


Data



Models

Key question: How to find performant programs for each operator?



```
C = tvm.compute((m, n),  
    lambda y, x: tvm.sum(A[k, y] * B[k, x], axis=k))
```

Search Space of Possible Program Optimizations

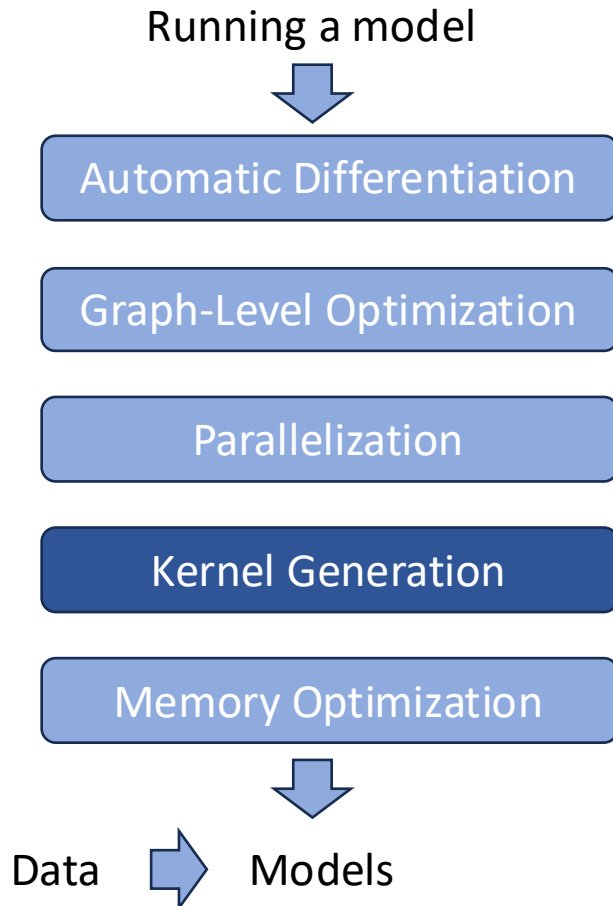
Low-level Program Variants

```
inp_buffer AL[8][8], BL[8][8]  
acc_buffer CL[8][8]  
for yo in range(128):  
    for xo in range(128):  
        vdl.fill_zero(CL)  
        for ko in range(128):  
            vdl.dma_copy2d(AL, A[ko*8:ko*8+8][yo*8:yo*8+8])  
            vdl.dma_copy2d(BL, B[ko*8:ko*8+8][xo*8:xo*8+8])  
            vdl.fused_gemm8x8_add(CL, AL, BL)  
            vdl.dma_copy2d(C[yo*8:yo*8+8,xo*8:xo*8+8], CL)
```

```
for yo in range(128):  
    for xo in range(128):  
        C[yo*8:yo*8+8][xo*8:xo*8+8] = 0  
        for ko in range(128):  
            for yi in range(8):  
                for xi in range(8):  
                    for ki in range(8):  
                        C[yo*8+yi][xo*8+xi] +=  
                            A[ko*8+ki][yo*8+yi] * B[ko*8+ki][xo*8+xi]
```

```
for y in range(1024):  
    for x in range(1024):  
        C[y][x] = 0  
        for k in range(1024):  
            C[y][x] += A[k][y] * B[k][x]
```

Kernel Generation



Possible solutions: Optimized Tensor Programs

Hardware vendors provide operator libraries manually developed by software/hardware engineers

- cuDNN, cuBLAS, cuRAND, cuSPARSE for GPUs
- cudnnConvolutionForward() for convolution
- cublasSgemm() for matrix multiplication

Memory Optimization

Running a model



Automatic Differentiation

Graph-Level Optimization

Parallelization

Kernel Generation

Memory Optimization

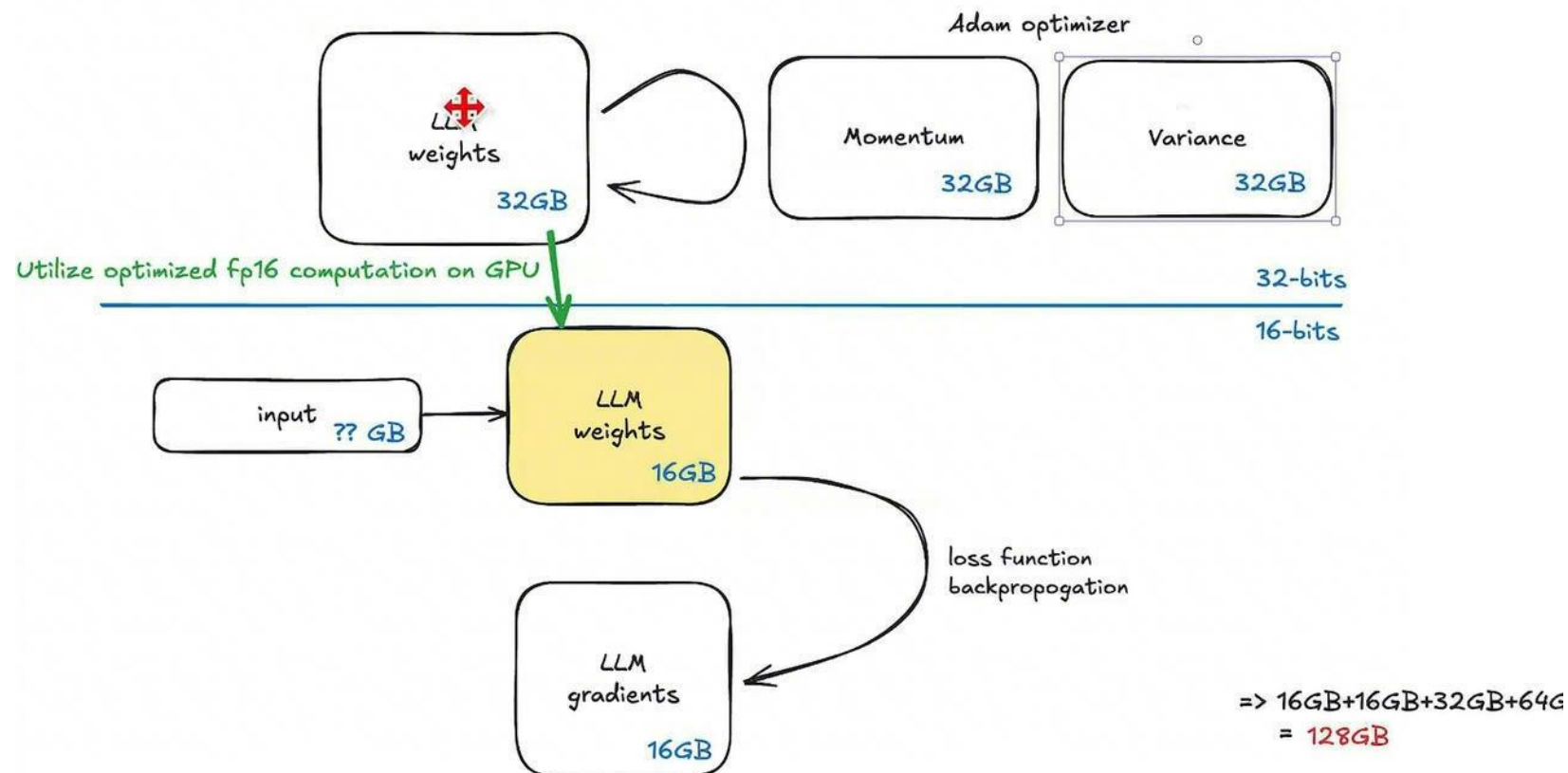
Data



Models

Why memory bound:

- fp32: 8B params = $8 * 10^9 * 32\text{bit} = 32\text{GB}$
- fp16: 8B params = $8 * 10^9 * 16\text{bit} = 16\text{GB}$



Thanks!